Data structures and algorithms

Instructor: Masoud Asghari

Web page: http://www.maser.ir/lectures/dsas2017sut/

Ch 0.2 (Preliminary Pointers)

In this Slide Series:

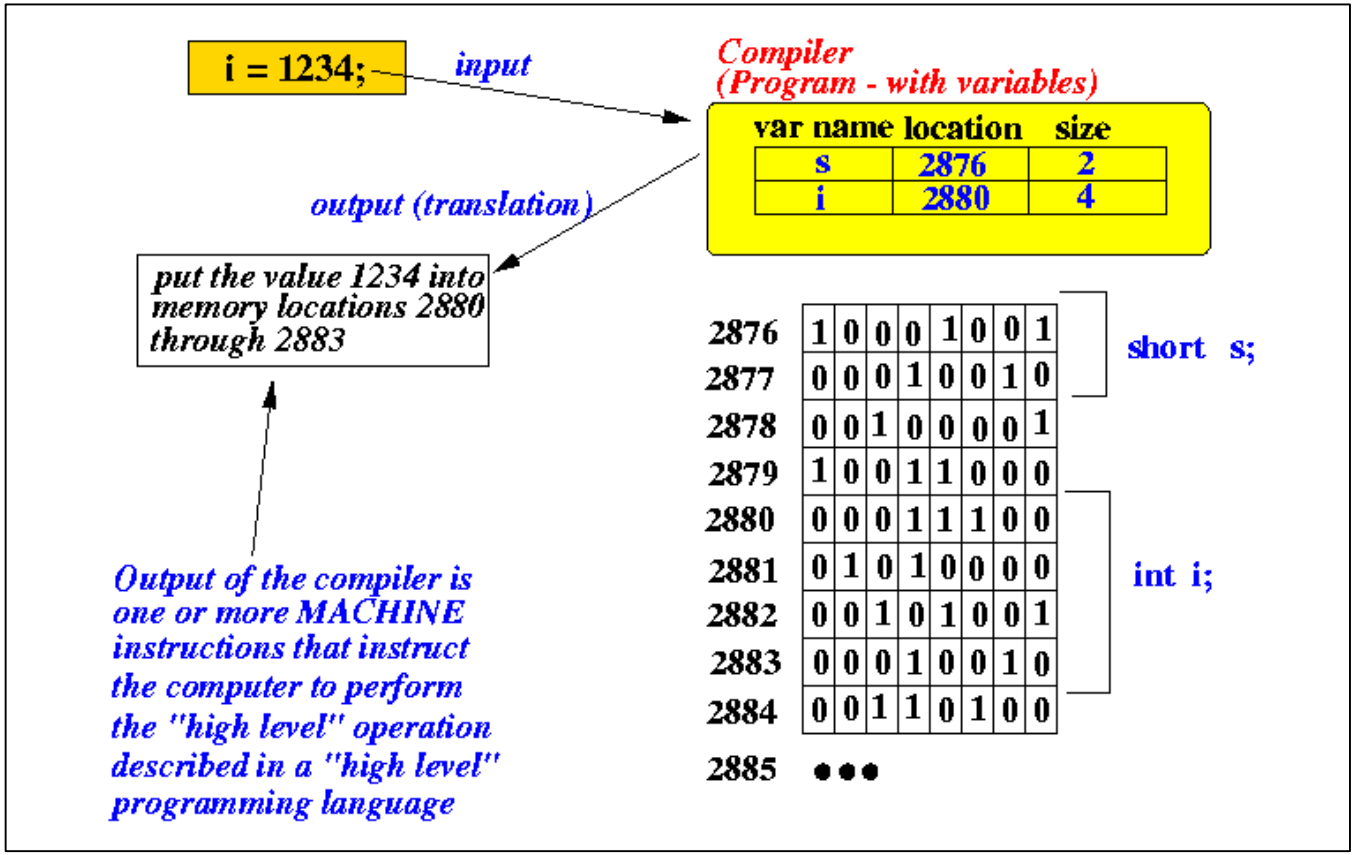
1. C++ Mainly from http://www.cplusplus.com/doc/tutorial/
   - Advanced
     - Pointers
       - Functions (Arguments passed by value and by reference)
     - Dynamic Memory
     - Data structures

Introduction

- variables have been explained as locations in the memory
  - which can be accessed by their identifier (their name).
  - This way, the program does not need to care about the physical address of the data in memory;
  - it simply uses the identifier whenever it needs to refer to the variable.
- the memory of a computer
  - is like a succession of memory cells,
  - each one byte in size, and
  - each with a unique address.
- data representations larger than one byte
  - Can occupy memory cells that have consecutive addresses
- each cell (byte of memory)
  - can be easily located in the memory by means of its unique address

Address	Contents
00000000	11100011
00000001	10101001
⋮	⋮
11111100	00000000
11111101	11111111
11111110	10101010
11111111	00110011

- When a variable is declared,
  - the memory needed to store its value is assigned a specific location in memory (its memory address).
- operating system
  - decides the particular memory locations on runtime.
  - However, it may be useful for a program to be able to obtain the address of a variable during runtime



Pointers

- **Address-of**
  - The address of a variable (beginning cell of where its value is saved)
  - can be obtained by preceding the name of a variable with an (&) sign
    - known as address-of operator.
- We need a variable who can save a physical address of a type.
  - This variable is called a **pointer**.
  - It only saves the address of a single memory cell (Byte)
  - To **declare a pointer** :

```
type * name;
```

    - type is the data type pointed to by the pointer.
    - This type is not the type of the pointer itself, but the type of the data the pointer points to.

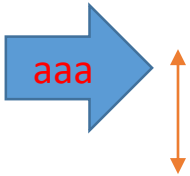
```
int a;        //a is an int type variable
int * b;      //b is a pointer, pointing to int types

b = &a;
```

Pointers

```
short aaa;
short * aaap;
aaa=1;
aaap= &aaa;
```

- How does above code work
  - Line1:
    - Your program asks O.S. for 2 consecutive bytes at memory (short type)
    - O.S. allocates 2 consecutive bytes and returns their first cell memory address
    - Assuming a 32 bit computer, the returned memory address is a 32bit memory address = 32/8= 4Byte address
    - Assuming a 64 bit computer, the returned memory address is a 64bit memory address = 64/8= 8Byte address
    - For example O.S. returns 0x00000ba0
    - Memory cells of (0x00000ba0 to 0x00000ba1) are reserved for **aaa** variable
    - From now on, wherever, you use **aaa** variable, the program replaces it with its memory address which is 0x00000ba0

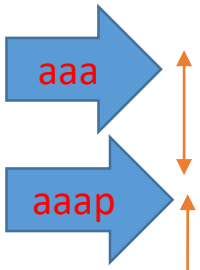


Address (4Byte)	Content (1Byte)
0x00000b9d	0x??
0x00000ba0	0x??
0x00000ba1	0x??
0x00000ba2	0x??

Pointers

```
short aaa;
short * aaap;
aaa=1;
aaap= &aaa;
```

- How does above code work
  - Line2:
    - Your program asks O.S. for 4 (or 8) consecutive bytes at memory depending on 32 bit (or 64 bit) computer architecture
    - These memory cells will be used to save address of other memory cells that contains short type value
    - O.S. allocates 4 (or 8) consecutive bytes depending on 32 bit (or 64 bit) computer architecture, and returns their first cell memory address
    - For example O.S. returns 0x00000ba2
    - Memory cells of (0x00000ba2 to 0x00000ba5) are reserved for **aaap** pointer
    - From now on, wherever, you use **aaap**, the program replaces it with its memory address which is 0x00000ba2

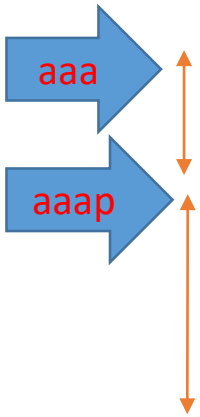


Address (4Byte)	Content (1Byte)
0x00000b9d	0x??
0x00000ba0	0x??
0x00000ba1	0x??
0x00000ba2	0x??
0x00000ba3	0x??
0x00000ba4	0x??
0x00000ba5	0x??
0x00000ba6	0x??

Pointers

```
short aaa;
short * aaap;
aaa=1;
aaap= &aaa;
```

- How does above code work
  - Line3:
    - Your program sees an assignment operation.
    - Right hand side is 0x0001
    - And the left hand side is a variable name aaa
    - As told, aaa is replaced with its memory address 0x00000ba0
    - Thus lines 3 says to computer that put 0x0001 in memory cells starting from 0x00000ba0 to 0x00000ba1 addresses
      - In particular, put 0x00 in 0x00000ba0 address
      - And put 0x01 in 0x00000ba1 address
      - This order changes depending on little or big Endian architecture



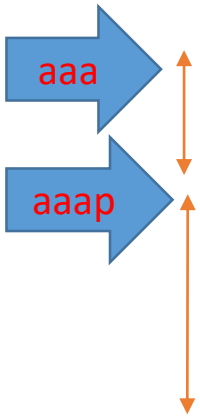
Address (4Byte)	Content (1Byte)
0x00000b9d	0x??
0x00000ba0	0x00
0x00000ba1	0x01
0x00000ba2	0x??
0x00000ba3	0x??
0x00000ba4	0x??
0x00000ba5	0x??
0x00000ba6	0x??

Pointers

```
short aaa;
short * aaap;
aaa=1;
aaap= &aaa;
```

- How does above code work
  - Line4:
    - Your program sees an assignment operation.
    - Right hand side is and address of operator followed by a variable
    - address of operator tells your program to reflect the actual address of the variable
    - Here, &aaa equals 0x00000ba0
    - And the left hand side is a pointer name aaap
    - As told, aaap is replaced with its memory address 0x00000ba2

    - Thus lines 4 says to computer that put 0x00000ba0 in memory cells starting from 0x00000ba2 to 0x00000ba5 addresses
      - As the figure shows



Address (4Byte)	Content (1Byte)
0x00000b9d	0x??
0x00000ba0	0x00
0x00000ba1	0x01
0x00000ba2	0x00
0x00000ba3	0x00
0x00000ba4	0x0b
0x00000ba5	0xa0
0x00000ba6	0x??

Pointers

- **Example 2**

```
1   //We have 64 bit CPU, = 8Byte address
2   #include <iostream>
3   using namespace std;
4
5   int main(int argc, char** argv) {
6       int a;
7       int * ap;
8       char b;
9       char * bp;
10      short c;
11      short * cp;
12
13      cout << sizeof(a) << '-'<< sizeof(ap) <<',';
14      cout << sizeof(b) << '-'<< sizeof(bp) <<',';
15      cout << sizeof(c) << '-'<< sizeof(cp) <<',';
16      //prints 4-8,1-8,2-8,
17
18  }
```

Pointers

- **Dereference operator (*)**
  - A pointers can be used to access the variable it points to directly.
  - This is done by preceding the pointer name with the dereference operator (*).
  - The operator itself can be read as "value pointed to by".
- The reference and dereference operators are thus complementary:
  - & is the address-of operator, and can be read simply as "address of"
  - * is the dereference operator, and can be read as "value pointed to by"

```
#include <iostream>
using namespace std;

int main(int argc, char** argv) {
    int a,b;
    int * ap;    //define a pointer

    a=123;
    b=0;
    ap = & a;    //get address of a
                 //ap points to a
    b= *ap;      //get value pointed by ap
    cout <<a<<'|'<<b<<'\n'; //123|123

    *ap = 99;    //set value pointed by ap
    cout <<a<<'|'<<b<<'\n'; //99|123

    ap = & b;    //get address of b
                 //ap points to b
    *ap = 88;    //set value pointed by ap
    cout <<a<<'|'<<b<<'\n'; //99|88

    cout << *(& a) <<'\n'; //99 equals to a

}
```

Pointers

- **Declaring pointers** `type * name;`
  - Why do we need a type for declaring a pointer?
    - Due to the ability of a pointer to directly refer to the value that it points to,
    - a pointer has different properties when it points to a char than when it points to an int or a float.
    - Once dereferenced, the type needs to be known.
    - And for that, the declaration of a pointer needs to include the data type the pointer is going to point to.

```
int * number;
char * character;
double * decimals;
```

  - the first one points to an int, the second one to a char, and the last one to a double.
    - Therefore, although these three example variables are all pointers,
    - they actually have different types: int*, char*, and double* respectively, depending on the type they point to

Pointers

- Note that the asterisk (*) used when declaring a pointer only means that it is a pointer (it is part of its type compound specifier)
  - This should not be confused with the dereference operator, which is also written with an asterisk (*).
  - They are two different things represented with the same sign.

```
// more pointers
#include <iostream>
using namespace std;

int main ()
{
  int firstvalue = 5, secondvalue = 15;
  int * p1, * p2;

  p1 = &firstvalue;  // p1 = address of firstvalue
  p2 = &secondvalue; // p2 = address of secondvalue
  *p1 = 10;          // value pointed to by p1 = 10
  *p2 = *p1;         // value pointed to by p2 = value pointed to by p1
  p1 = p2;           // p1 = p2 (value of pointer is copied)
  *p1 = 20;          // value pointed to by p1 = 20

  cout << "firstvalue is " << firstvalue << '\n';
  cout << "secondvalue is " << secondvalue << '\n';
  return 0;
}
```

```
firstvalue is 10
secondvalue is 20
```

Pointer arithmetic

- only addition and subtraction operations are allowed on pointers
  - both addition and subtraction have a different behavior with pointers, according to the size of the data type to which they point.
  - We know that different types have different sizes.
    - the exact size of different types, dependents on the system.
    - For example, assume, char takes 1 byte, short takes 2 bytes, and long takes 4.
    - Suppose now that we define three pointers:

```
char *mychar;
short *myshort;
long *mylong;
```

    - and they point to the memory locations 1000, 2000, and 3000, respectively.
    - if we write:

```
++mychar;
++myshort;
++mylong;
```

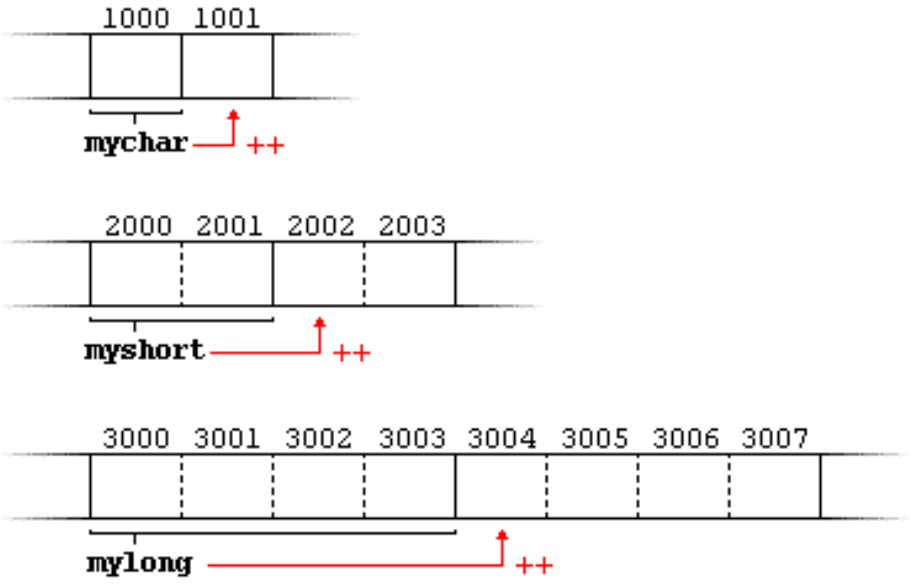
    - Or

```
mychar = mychar + 1;
myshort = myshort + 1;
mylong = mylong + 1;
```

    - mychar would contain the value 1001.
    - But myshort would contain the value 2002, and mylong would contain 3004, even though they have each been incremented by one.
  - The reason is that, when adding one to a pointer, the pointer is made to point to the element of its type, and, therefore, the size in bytes of the type it points to is added to the pointer.

Pointer arithmetic

```
char *mychar;
short *myshort;
long *mylong;
```



- Remembering operator precedence rules,
  - postfix operators, such as increment and decrement, have higher precedence than prefix operators, such as the dereference operator (*).

```
*p++   // same as *(p++): increment pointer, and dereference unincremented address
*++p   // same as *(++p): increment pointer, and dereference incremented address
++*p   // same as ++(*p): dereference pointer, and increment the value it points to
(*p)++ // dereference pointer, and post-increment the value it points to
```

  - Parentheses reduce confusion by adding legibility to expressions

Pointers and arrays

- The concept of arrays is related to that of pointers.
  - In fact, arrays work very much like pointers to their first elements,
  - brackets [ ] were explained as specifying the index of an element of the array.
    - in fact They dereference the variable they follow just as * does, but they also add the number between brackets to the address being dereferenced.
  - These expressions are equivalent and valid not only if a is a pointer, but also if a is an array

```
a[5] = 0;       // a [offset of 5] = 0
*(a+5) = 0;     // pointed to by (a+5) = 0
```

  - an array can always be implicitly converted to the pointer of the proper type.
  - For example:

```
int myarray [20];
int * mypointer;
```

    - The following assignment operation would be valid:

```
mypointer = myarray;
```

    - After that, mypointer and myarray would be equivalent and would have very similar properties.
    - The **main difference** is that mypointer can be assigned a different address, whereas myarray can never be assigned anything, and will always represent the same block of 20 elements of type int.

Pointers and arrays

- an example that mixes arrays and pointers

```
// more pointers
#include <iostream>
using namespace std;

int main ()
{
  int numbers[5];
  int * p;
  p = numbers;  *p = 10;
  p++;  *p = 20;
  p = &numbers[2];  *p = 30;
  p = numbers + 3;  *p = 40;
  p = numbers;  *(p+4) = 50;
  for (int n=0; n<5; n++)
    cout << numbers[n] << ", ";
  return 0;
}
```

```
10, 20, 30, 40, 50,
```

cout<< p[n] << “, “; //the same
cout<< *(p+n) << “, “; //the same
cout<< *(numbers+n) << “, “; //the same

- Pointers and arrays support the same set of operations, with the same meaning for both.
- The main difference is that pointers can be assigned new addresses, while arrays cannot.
- Also, arrays allocate memory at the time of definition (with fixed amount), but pointers need allocated memory to point to.

Pointers and string literals

Very important

- Problem with array of characters (Slide 2.53)
  - once myword has already been declared as below

```
char myword[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
char myword[] = "Hello";
```

  - Its sequence of values cannot be changed as above.
  - Only value of each single element can be changed

```
myword[0] = 'B';
myword[1] = 'y';
myword[2] = 'e';
myword[3] = '\0';
```

- Using pointers as a solution
  - Remember: pointers can be assigned new addresses, while arrays cannot

Pointers and string literals

- *string literals*
  - are arrays containing null-terminated character sequences.
  - When you write `cout<<"Hi";`
    - You define an array of chars with 3 elements: {'H', 'i', '\n'}
    - So basicly "Hi" returns a pointer (array) pointing to its first element (containing address of the first element in the memory)
    - This address then is used by cout to get the characters
  - cout accepts following as a strings:
    - *string literals* such as "Hi"
      - *A char such as* 'H'
    - Name of array of chars such as x that defined as: char x[10];
      - An element of this array such as x[2]
    - Name of pointer of type char* such as p that defined as: char* p;
      - A single element of this pointer such as *(p+2)
  - cin accepts following as a strings:
    - Name of array of chars (replaces its elements value)
    - Name of pointer of type char* (memory should be allocated first)

Pointers and string literals

- Example

```
1   #include <iostream>
2   using namespace std;
3
4   int main(int argc, char** argv) {
5       char* a;
6       char b[]="wait";
7
8       a="hi"; //implicit type casting
9               //warning of deprecated conversion
10      cout<<a<<'|'<<b<<'\n'; //hi|wait
11      cout<<*(a+1)<<'|'<<*(b+1)<<'\n'; //i|a
12
13      a=(char*)"hello"; //explicit type casting
14      cout<<a<<'|'<<b<<'\n'; //hello|wait
15
16      a=b;
17      cout<<a<<'|'<<b<<'\n';//wait|wait
18
19      cin>>a; //bye
20      cout<<a<<'|'<<b<<'\n';//bye|bye
21
22      cin>>a; //test1
23      //OS. out of range error
24      //a points to b which allocated only 4char
25
26  }
```

Pointers and const

- Pointers can be used to access a variable by its address,
  - this access may include modifying the value pointed.
  - But it is possible to declare pointers that can access the pointed value to read it, but not to modify it.
  - For this, it is enough with qualifying the type pointed to by the pointer as const.
  - For example:

```
int x;
int y = 10;
const int * p = &y;
x = *p;          // ok: reading p
*p = x;          // error: modifying p, which is const-qualified
```

  - Pointers can also be themselves const.
  - And this is specified by appending const to the pointed type (after *)

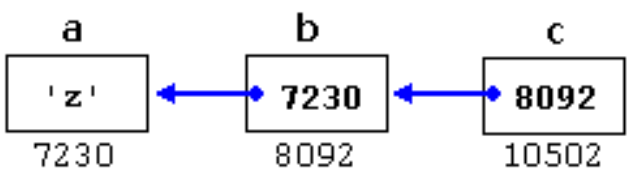
```
int x;
      int *       p1 = &x;  // non-const pointer to non-const int
const int *       p2 = &x;  // non-const pointer to const int
      int * const p3 = &x;  // const pointer to non-const int
const int * const p4 = &x;  // const pointer to const int
```

Pointers to pointers

C++ allows the use of pointers that point to pointers, that these, in its turn, point to data (or even to other pointers). The syntax simply requires an asterisk (`*`) for each level of indirection in the declaration of the pointer:

```
char a;
char * b;
char ** c;
a = 'z';
b = &a;
c = &b;
```

This, assuming the randomly chosen memory locations for each variable of `7230`, `8092`, and `10502`, could be represented as:



With the value of each variable represented inside its corresponding cell, and their respective addresses in memory represented by the value under them.

The new thing in this example is variable `c`, which is a pointer to a pointer, and can be used in three different levels of indirection, each one of them would correspond to a different value:

- `c` is of type `char**` and a value of `8092`
- `*c` is of type `char*` and a value of `7230`
- `**c` is of type `char` and a value of `'z'`

void pointers

- The void type of pointer is a special type of pointer.
  - void represents the absence of type.
  - Therefore, void pointers are pointers that point to a value that has no type
    (and thus also an undetermined length and undetermined dereferencing properties).
  - This gives void pointers a great flexibility:
    - by being able to point to any data type, from an integer value or a float to a string of characters.
  - This gives void pointers a great limitation:
    - the data pointed to by them cannot be directly dereferenced
    - (since we have no type to dereference to),
    - Thus any address in a void pointer needs to be transformed into some other pointer type that points to a concrete data type before being dereferenced.
    - This can be done by explicit type casting

```
#include <iostream>
using namespace std;

int main(int argc, char** argv) {
    char a = 'x';
    int b = 1602;
    void* data; //null pointer

    data= &a;
    *(char*)data='y';

    data= &b;
    *(int*)data=100;

    cout <<a<<'|'<<b
    <<'|'<<*(char*)data<<'\n';
    //y|100|d
}
```

Invalid pointers and null pointers

- Pointers can actually point to any address,
  - including addresses that do not refer to any valid element.
  - examples are uninitialized pointers and pointers to nonexistent elements of an array:

```
int * p;                // uninitialized pointer (local variable)

int myarray[10];
int * q = myarray+20;   // element out of bounds
```

  - Neither p nor q point to addresses known to contain a value, but none of the above statements causes an error.
  - What can cause an error is to dereference such a pointer. Accessing to the place that such a pointer points, causes undefined behavior, ranging from an error during runtime to accessing some random value.
- If you want to explicitly point nowhere,
  - a special value that any pointer type can take: the null pointer value.
  - null pointer value can be expressed by 0 or NULL:

```
int * p;
p = 0;
p = NULL;
```

Functions (Arguments passed by value and by reference)

- In the functions seen earlier, arguments have been passed *by value*.
  - when calling a function,
  - what is passed to the function are the values of these arguments on the moment of the call,
  - which are copied into the variables represented by the function parameters.
  - any modification of these variables within the function has no effect on the values of the original variables outside it
- arguments can be passed *by reference*, instead of *by value*.
  - Changes on parameters inside the function also changes the value of arguments calling it from the outside of function
- **How pass arguments by reference to an function**
  - You can define function's parameters as pointers and pass the address of arguments when calling the function
  - Or declares function's parameters as *references*.
    - **references** are indicated with an (&) following the parameter type
    - Call the function as usual

Functions (Arguments passed by value and by reference)

Examples of three methods of passing arguments

```
// passing parameters by value
#include <iostream>
using namespace std;

void duplicate (int a, int b, int c)
{
  a*=2;
  b*=2;
  c*=2;
}

int main ()
{
  int x=1, y=3, z=7;
  duplicate (x, y, z);
  cout << x << "," << y << "," << z;
  //1,3,7
  return 0;
}
```

```
// passing parameters by reference
#include <iostream>
using namespace std;

void duplicate (int& a, int& b, int& c)
{
  a*=2;
  b*=2;
  c*=2;
}

int main ()
{
  int x=1, y=3, z=7;
  duplicate (x, y, z);
  cout << x << "," << y << "," << z;
  //2,6,14
  return 0;
}
```

```
// passing parameters by pointers
#include <iostream>
using namespace std;

void duplicate (int* a, int* b, int* c)
{
  (*a) *=2;
  (*b) *=2;
  (*c) *=2;
}

int main ()
{
  int x=1, y=3, z=7;
  duplicate (&x, &y, &z);
  cout << x << "," << y << "," << z;
  //2,6,14
  return 0;
}
```

Functions (Arguments passed by value and by reference)

- Remember, arrays are in fact references
  - So you should declares function's parameters as pointers to pass an array or a pointer
  - Call with only name of array or pointer as arguments

```
#include <iostream>
using namespace std;

void somefunc (char* a1, char** a2,char* a3, int* a4)
{
  cout<<a1<<','<<*a2<<','<< a3<<','<< (*a4)<<'\n';
  a1[0]='A';
  *a2="four";
  a3="five";
  *a4=19;
}

int main (){
  char s1[]="One";
  char* s2 ="Two";
  char* s3 ="Three";
  int a=20;
  int* ap;
  ap =&a;
  somefunc(s1,&s2,s3,ap);

  somefunc(s1,&s2,s3,ap);
  return 0;
}
```

```
One,Two,Three,20
Ane,four,Three,19

--------------------------------
```

Dynamic memory

- When declaring primitive types or arrays
  - Compiler allocates memory for them
  - This memory allocation is static
- As you have seen, pointers have no where to point at the declaration
  - They so far referred to already allocated memories (static)
- To allocate memory at runtime (dynamically)
  - We need memory allocation operators
  - and to handle them, **we use pointers**
  - Examples: unknown length of arrays before running
- You need to include <new> library at the beginning

```
#include <new>
```

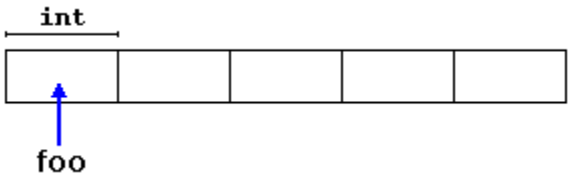
Dynamic memory

- **Operators new and new[]**
  - Dynamic memory is allocated using operator new.
  - new is followed by a data type specifier and,
  - if a sequence of more than one element is required, the number of these within brackets [ ].
  - It returns a pointer to the beginning of the new block of memory allocated.
  - Its syntax is:

```
pointer = new type
pointer = new type [number_of_elements]
```

  - Example:

```
int * foo;
foo = new int [5];
```



    - the system dynamically allocates space for five elements of type int
    - and returns a pointer to the first element of the sequence, which is assigned to foo (a pointer).
    - Therefore, foo now points to a valid block of memory with space for five elements of type int.

Dynamic memory

- **Operators delete and delete[]**
  - once the memory is no longer needed,
  - it can be freed so that the memory becomes available again for other requests of dynamic memory.
  - Also if you know that the block of memory will not be pointed
    - You should free (delete) it to prevent **memory leaks**

  - Dynamic memory is freed using operator delete.
  - delete is followed by a pointer,
  - if a sequence of more than one element is allocated by new, use [ ] after delete.
  - Its syntax is:

```
delete pointer;
delete[] pointer;
```

Dynamic memory

- **Example**

```
#include <iostream>
#include <new>
using namespace std;

int main (){
    int i,n;
    int * p;
    cout << "How many numbers would you like to type? ";
    cin >> i;
    p= new int[i];
    for (n=0; n<i; n++)
    {
        cout << "Enter number: ";
        cin >> p[n];
    }
    cout << "You have entered: ";
    for (n=0; n<i; n++)
      cout << p[n] << ", ";
    delete[] p;

  return 0;
}
```

```
How many numbers would you like to type? 5
Enter number : 75
Enter number : 436
Enter number : 1067
Enter number : 8
Enter number : 32
You have entered: 75, 436, 1067, 8, 32,
```

Dynamic memory in C

- C++ integrates the operators new and delete for allocating dynamic memory.
  - But these were not available in the C language;
  - instead, it used a library solution, with the functions malloc, calloc, realloc and free,
  - defined in the header <cstdlib> (known as <stdlib.h> in C).
  - The functions are also available in C++ and can also be used to allocate and deallocate dynamic memory.

Dynamic memory in C

- **malloc** `void* malloc (size_t size);`
  - Allocates a block of *size* bytes of memory,
  - returns a pointer to the beginning of block.
    - size_t is an unsigned integral type.
  - **Return Value**
    - On success, a pointer to the memory block allocated by the function.
    - The type of this pointer is always void*, which can be cast to the desired type of data pointer in order to be dereferenceable.
    - If the function failed to allocate the requested block of memory, a null pointer is returned.

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main (){
    int i,n;
    int * p;
    cout << "How many numbers would you like to type? ";
    cin >> i;
    p= (int*) malloc(i*sizeof(int));
    for (n=0; n<i; n++)
    {
        cout << "Enter number: ";
        cin >> p[n];
    }
    cout << "You have entered: ";
    for (n=0; n<i; n++)
      cout << p[n] << ", ";

    free(p);

  return 0;
}
```

Dynamic memory in C

- http://www.cplusplus.com/reference/cstdlib/calloc/

calloc

<cstdlib>

```
void* calloc (size_t num, size_t size);
```

Allocate and zero-initialize array

Allocates a block of memory for an array of *num* elements, each of them *size* bytes long, and initializes all its bits to zero.

The effective result is the allocation of a zero-initialized memory block of `(num*size)` bytes.

- http://www.cplusplus.com/reference/cstdlib/realloc/

realloc

<cstdlib>

```
void* realloc (void* ptr, size_t size);
```

Reallocate memory block

Changes the size of the memory block pointed to by *ptr*.

The function may move the memory block to a new location (whose address is returned by the function).

The content of the memory block is preserved up to the lesser of the new and old sizes, even if the block is moved to a new location. If the new *size* is larger, the value of the newly allocated portion is indeterminate.

In case that *ptr* is a null pointer, the function behaves like `malloc`, assigning a new block of *size* bytes and returning a pointer to its beginning.

If the function fails to allocate the requested block of memory, a null pointer is returned, and the memory block pointed to by argument *ptr* is not deallocated (it is still valid, and with its contents unchanged).

Dynamic memory in C

- **free**

```
void free (void* ptr);
```

  - **Deallocates memory block**
  - with a given pointer to block of memory previously allocated by a call to malloc, calloc or realloc is deallocated, making it available again for further allocations.
  - If ptr does not point to a block of memory allocated with the above functions, it causes undefined behavior.
  - Notice that this function does not change the value of ptr itself, hence it still points to the same (now invalid) location.

```c
/* free example */
#include <stdlib.h>     /* malloc, calloc, realloc, free */

int main ()
{
  int * buffer1, * buffer2, * buffer3;
  buffer1 = (int*) malloc (100*sizeof(int));
  buffer2 = (int*) calloc (100,sizeof(int));
  buffer3 = (int*) realloc (buffer2,500*sizeof(int));
  free (buffer1);
  free (buffer3);
  return 0;
}
```

Data structures

- A *data structure*
  - User defined data_type (like int, double, … you define yours)
  - is a group of data elements grouped together under one name.
  - These data elements, known as *members*, can have different types and different lengths.
  - Declaration syntax:
    - type_name is a name for the structure type,
    - Within braces { }, there is a list with the data members, each one is specified with a type and a valid identifier as its name.

```
struct type_name {
member_type1 member_name1;
member_type2 member_name2;
member_type3 member_name3;
.
.
} ;
```

  - After defining a struct (your data type), you can declare variables of this newly defined type (As usual variables)
  - Example:

```
struct product {
  int weight;
  double price;
} ;

product apple;
product banana, melon;
```

Data structures

```
struct product {
  int weight;
  double price;
} ;

product apple;
product banana, melon;
```

- It is important to clearly differentiate between
  - what is the structure type name (product),
  - and what is an object of this type (apple, banana, and melon).
  - Many objects (such as apple, banana, and melon) can be declared from a single structure type (product).
- Once an objects of a determined structure type is declared
  - its members can be accessed directly by inserting a dot (.) between the object name and the member name.
  - We could operate with any of these elements as if they were standard variables of their respective types
- one of the features of data structures
  - is the ability to refer to both their members individually
  - or to the entire structure as a whole.
  - In both cases using the same identifier
    - (the name of the structure).

```
apple.weight
apple.price
banana.weight
banana.price
melon.weight
melon.price
```

Data structures Example

```
#include <iostream>
#include <new>
using namespace std;

struct movies_t {
  char * title;
  int year;
};

void printmovie (movies_t movie){
  cout << movie.title;
  cout << " (" << movie.year << ")\n";
}

int main (){
  movies_t mine, yours;
  mine.title = "2001 A Space Odyssey";
  mine.year = 1968;

  yours.title=new char[100];
  cout << "Enter title: (no space) ";
  cin >> yours.title;
  cout << "Enter year: ";
  cin >> yours.year;

  cout << "My favorite movie is:\n ";
  printmovie (mine);
  cout << "And yours is:\n ";
  printmovie (yours);
  return 0;
}
```

```
struct movies_t {
  char * title;
  int year;
};

void printmovie (movies_t& movie){
  cout << movie.title;
  cout << " (" << movie.year << ")\n";
}

int main (){
  movies_t mine, yours;
  mine.title = "2001 A Space Odyssey";
  mine.year = 1968;

  yours.title=new char[100];
  cout << "Enter title: (no space) ";
  cin >> yours.title;
  cout << "Enter year: ";
  cin >> yours.year;

  cout << "My favorite movie is:\n ";
  printmovie (mine);
  cout << "And yours is:\n ";
  printmovie (yours);
  return 0;
}
```

Pointers to structures

- Like any other type,
  - structures can be pointed to by its own type of pointers:

```
struct movies_t {
  string title;
  int year;
};

movies_t amovie;
movies_t * pmovie;
```

  - amovie is an object of structure type movies_t,
  - pmovie is a pointer to point to objects of structure type movies_t.
  - Therefore, the following code would also be valid:

```
pmovie = &amovie;
```

  - The value of the pointer pmovie would be assigned the address of object amovie.

Pointers to structures

- The arrow operator (->)
  - is a dereference operator that is used exclusively with pointers to objects that have members
  - (*pointer_to_an_object).member_name
  - equals to
  - pointer_to_an_object-> member_name

    - Example: Given

```
movies_t amovie;
movies_t * pmovie;
pmovie = &amovie;
```

    - pmovie->title is equivalent to (*pmovie).title
    - Both has the value of amovie.title

Expression	What is evaluated	Equivalent
a.b	Member b of object a	
a->b	Member b of object pointed to by a	(*a).b
*a.b	Value pointed to by member b of object a	*(a.b)

Pointers to structures

- **Nesting structures**

Structures can also be nested in such a way that an element of a structure is itself another structure:

```
struct movies_t {
  string title;
  int year;
};

struct friends_t {
  string name;
  string email;
  movies_t favorite_movie;
} charlie, maria;

friends_t * pfriends = &charlie;
```

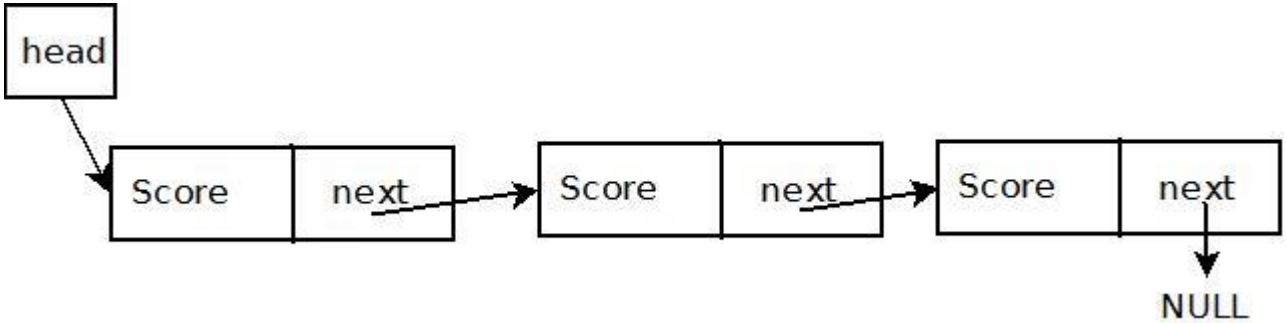
After the previous declarations, all of the following expressions would be valid:

```
charlie.name
maria.favorite_movie.title
charlie.favorite_movie.year
pfriends->favorite_movie.year
```

(where, by the way, the last two expressions refer to the same member).

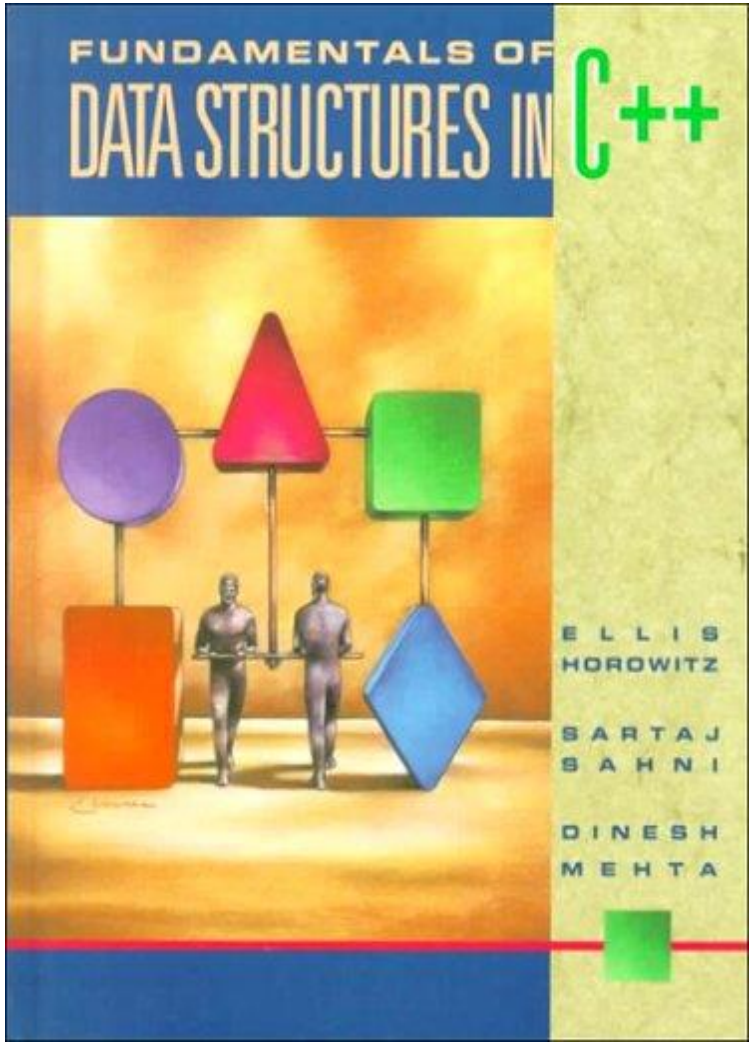
Linked Lists (dynamic memory) by struct

```
1   #include <iostream>
2   #include <new>
3   using namespace std;
4
5   struct node_t {
6     int score;
7     node_t * next=NULL;
8   };
9
10  int main (){
11    node_t * head_node=NULL;
12    node_t * current_node=NULL;
13    node_t * prev_node=NULL;
14    bool is_head=true;
15    char is_continue;
16
17    while(true){
18      prev_node=current_node;
19      current_node = new node_t;
20      if(is_head){
21          head_node=current_node;
22          is_head=false;
23      }else{
24          prev_node->next=current_node;
25      }
26
27      cout << "Enter Score: (0 to stop): ";
28      cin >> current_node->score;
29      if(current_node->score == 0)break;
30    }
```




```
31    //done, now show the list
32    cout << "Your Scores are: ";
33    current_node=head_node;
34    while(true){
35      cout << current_node->score <<", ";
36      if(current_node->next ==NULL)break;
37      current_node = current_node->next;
38    }
39    //current_node points to the last node
40    return 0;
41  }
```

Back to the BOOK



```
int   i=25
int   *np ;
np  =  &i ;
```

اشاره گرها : آدرس اشیاء را در خود نگهداری می‌کنند، ب

Cout<< * np; // de reference operator

```
int i = 5 ;
int& j = i ;
i = 7 ;
```

۶) نوع مرجع (reference type)

یک روش برای برخورداری از مزیت هر دو روش انتقال پارامتر این است که مرجع‌های ثابت (constant references) مانند const T&a (T نوع آرگومان است) را منتقل کنیم.

Page 16, 20 of the book

یک استثناء برای این قاعده که روش پیش فرض انتقال پارامترها در C++ انتقال به وسیله مقدار است، وجود دارد و آن استثناء این است که نوع آرایه با آدرسش منتقل می‌شود؛ بدین معنی که آرایه در حافظه محلی تابع کپی نمی‌شود. بنابراین هر تغییری که در داخل تابع بر روی آرایه صورت گیرد در خود آرایه اصلی نیز اتفاق می‌افتد. وقتی یک تابع با یک آرگومان آرایه a (مانند $f(a)$) فراخوانی می‌شود، در حقیقت اشاره گری به اولین عنصر a (یعنی، $\&a[0]$) منتقل می‌شود. علت آن این است که در تعاریف تابع، آرایه‌ها معمولاً با اشاره گرهایی به یک نوع مشخص می‌شوند (به عنوان مثال، **f(int *a)**). چون آرایه‌ها به این روش منتقل می‌شوند، لذا تابع اندازه آرایه‌ها را نمی‌تواند تشخیص دهد. این مشکل به راحتی با انتقال اندازه آرایه به عنوان یک پارامتر جدا به تابع حل می‌شود.

۹-۴-۱ سربارگذاری نام تابع در C++

overloading

آخرین بحث ما راجع به سربارگذاری نام تابع در C++ است. سربارگذاری تابع به این معناست که می‌توان چندین تابع با نام‌های یکسان با آرگومان‌های متفاوت داشت. برای مثال تمام توابع زیر با اینکه نام همه آنها *max* است در C++ اجازه همزیستی دارند.

```
int max(int, int);
int max(int, int, int);
int max(int*, int);
int max(float, int);
int max(int, float);
```


Page 20 of the book

تخصیص حافظه پویا در C++

عملگر new یک شیء را از نوع داده مطلوب ایجاد می‌کند و اشاره‌گری به نوع داده آن را بر می‌گرداند.
اگر قادر به ایجاد شیء مطلوب نباشد، مقدار صفر را بر می‌گرداند.

```
int *ip = new int;
if (ip == 0) cerr << "Memory not allocated" << endl ;
.
.
delete ip ;
```

```
int *jp = new int[10] ; // jp is an array of integers
if (jp == 0) cerr << "Memory not allocated" << endl ;
.
.
delete [ ] jp ; // delete the array jp
```

Classes

Classes are an expanded concept of *data structures*: like data structures, they can contain data members, but they can also contain functions as members.

An *object* is an instantiation of a class. In terms of variables, a class would be the type, and an object would be the variable.

Classes have the same format as plain *data structures*, except that they can also include functions and have these new things called *access specifiers*. An *access specifier* is one of the following three keywords: `private`, `public` or `protected`. These specifiers modify the access rights for the members that follow them:

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
// In the header file Rectangle.h
class Rectangle {
public:        // the following members are public
   // The next four members are member functions
   Rectangle();      // constructor
   ~Rectangle();     // destructor
   int GetHeight();  // returns the height of the rectangle
   int GetWidth();   // returns the width of the rectangle
private:          // the following members are private
   // the following members are data members
   int x1, y1, h, w;
   // (x1, y1) are the coordinates of the bottom left corner of the rectangle
   // w is the width of the rectangle; h is the height of the rectangle
};
#endif
```

برنامه ۱-۲. تعریف کلاس *Rectangle* در *C++*

پیاده‌سازی توابع، در یک فایل برنامه با

همان نام قرار می‌گیرد (به عنوان مثال، محتویات برنامه ۲-۲ در فایل *Rectangle.C* قرار گرفته است). در اینجا دقت داشته باشید که قواعد و ساخت نحوی زبان C++ به شما اجازه می‌دهد که پیاده‌سازی یک تابع عضو را در داخل تعریف کلاسش انجام دهید. در این حالت، تابع به عنوان یک تابع درون‌خطی (*inline*) تلقی می‌شود.

```
// In the source file Rectangle.C
#include "Rectangle.h"

// The prefix "Rectangle::" identifies GetHeight() and GetWidth() as member functions
// belonging to class Rectangle. It is required because the member functions
// are implemented outside their class definition

int Rectangle::GetHeight() { return h;}
int Rectangle::GetWidth() { return w;}
```

برنامه ۲-۲. پیاده‌سازی اعمال بر روی *Rectangle*

۳-۱-۲ تعریف (اعلان) اشیاء کلاس و فراخوانی توابع عضو

اشیاء کلاس به همان شیوه تعریف یا ایجاد متغیرها بیان و ایجاد می‌شوند.

عضوهای یک شی با استفاده از عملگرهای انتخاب یعنی یک نقطه (.) برای انتخاب مستقیم جزء و نماد پیکان (< -)، که ما به صورت (→) نمایش می‌دهیم ، برای انتخاب غیر مستقیم جزء به وسیله اشاره‌گر، قابل دسترسی یا فراخوانی هستند.

```
main() {
  Rectangle r, s;     // r and s are objects of class Rectangle
  Rectangle *t = &s;  // t is a pointer to class object s
  .
  .
  // use · to access members of class objects.
  // use → to access members of class objects through pointers.
  if (r.GetHeight () * r.GetWidth () > t →GetHeight () * t →GetWidth ())
    cout << " r ";
  else cout << " s ";
  cout << "has the greater area" << endl;
}
```

برنامه ۳-۲ . قطعه برنامه C++ که چگونگی تعریف اشیاء *Rectangle* و هم چنین فراخوانی توابع عضو را نشان می‌دهد.

۴-۱-۲ عملیات خاص روی کلاس‌ها

سازنده‌ها و مخرب‌ها: سازنده‌ها و مخرب‌ها توابع عضو ویژه از یک کلاس هستند. یک سازنده، تابع عضوی است که عضوهای داده یک شیء را مقدار دهی اولیه می‌کند. اگر برای یک کلاس، سازنده تعریف شده باشد در هنگام ایجاد شیئی از این کلاس این سازنده به طور خودکار اجرا می‌شود.

یک سازنده باید به عنوان یک تابع عضو عمومی کلاس خود تعریف شود؛ نام یک سازنده باید همنام کلاسی که به آن تعلق دارد، باشد؛ و یک سازنده نباید یک نوع بازگشتی را معین کند یا مقداری را برگرداند. برنامه ۴-۲ تعریف یک سازنده برای کلاس *Rectangle* را نشان می‌دهد.

```
Rectangle::Rectangle( int x, int y, int height, int width)
{
  x1 = x ; y1 = y ;
  h = height ; w = width ;
}
```

برنامه ۴-۲ . تعریف یک سازنده برای کلاس *Rectangle*

سازنده‌ها ممکن است برای مقدار دهی اشیاء کلاس *Rectangle* به صورت زیر استفاده شوند:

```
Rectangle r (1, 3, 6, 6);
Rectangle * s = new Rectangle (0, 0, 3, 4);
```

Exercises

1. Rewrite program in P31 as follows:
   - Don't ask for number of numbers
   - Get numbers as entered, and finish getting numbers if 0 is entered.
   - There is no limit for the number of numbers
   - You can use realloc() or any methods
2. Rewrite program in P42 as follows:
   - Each node_t includes (node_t * previous) pointer, links back to its parent
     - The first node's previous points to NULL
   - Add an extra loop after loop of line 31 to 39 finishes
     - Starting from the last node
     - Show list scores in reverse order